

REPRESENTACIÓN DE DATOS

Según ya vimos, las limitaciones de la electrónica convencional imponen una sola forma práctica de representación: la lógica de dos estados: “0” y “1”.

Un circuito electrónico digital conoce habitualmente dos estados: *apagado/prendido*, *conectado/desconectado*, los cuales se corresponden con las representaciones lógicas “0” y “1”.

La “representación binaria de datos” significa representar datos por medio de estos ceros y unos, denominados “dígitos binarios” o “bits” (contracción de binary digits).

Cualquier dato puede ser representado mediante una lógica binaria. Por ejemplo, el Código Morse utilizado en telegrafía, consiste en puntos y rayas; solo existe una posible elección binaria. Combinando estos puntos y rayas se pueden representar, después, números, caracteres, etcétera.

El bit es la unidad mínima de información. El problema a resolver es cómo los bits pueden representar los datos de una manera útil y cómo recuperar, posteriormente, esos datos.

Unidades de medida

8 bits = 1 byte

1024 bytes = 1 kilobyte

1024 kilobytes = 1 megabyte

1024 megabytes = 1 gigabyte

1024 gigabytes = 1 terabyte

1024 terabytes = 1 petabyte

1024 petabytes = 1 exabyte

1024 exabytes = 1 zettabyte

1024 zettabytes = 1 yottabyte

El método que se emplea en computación es el siguiente: dividir la memoria de la máquina en celdas de longitud fija y predeterminada de bits. Utilizando combinaciones de bits se pueden representar los símbolos deseados. A cada grupo de bits se le asigna una posición dentro de la memoria; es decir, una dirección. Esto permitirá encontrar dicho grupo cada vez que se necesite procesarlo.

Si vamos a representar datos de forma normal y que resulte comprensible, necesitaremos grupos de bits que identifiquen al menos lo siguiente:

- Los números del “0” al “9” (10 grupos de bits).
- Las letras del alfabeto (26 grupos de bits).
- Otros símbolos: { \$, #, &, *, @, %, ^, <, = }.

Harán falta por lo menos 36 combinaciones de bits para representar las letras y números:

- 1 bit da dos combinaciones: {0, 1}, 2^1 combinaciones.
- 2 bits dan cuatro combinaciones: {00, 01, 10, 11}, 2^2 combinaciones.
- 3 bits dan ocho combinaciones: {000, 001, 010, 011, 100, 101, 110, 111}, 2^3 combinaciones.

De esta manera, utilizando grupos de 6 bits se obtienen $2^6 = 64$ combinaciones posibles de bits que nos permiten representar 10 números, 26 letras, 28 símbolos. Así, durante cierto tiempo se utilizaron códigos de 6 bits, a ese conjunto se le llamó carácter.

Todo esto significa que cuando los datos externos son transmitidos a la computadora deben ser, primeramente, convertidos a una “representación interna” para que el computador pueda procesarlos. Después de que este proceso es llevado a cabo, los datos deben ser convertidos nuevamente a una “representación externa” para que el usuario reciba datos provistos de significado. Ver figura 4.1.

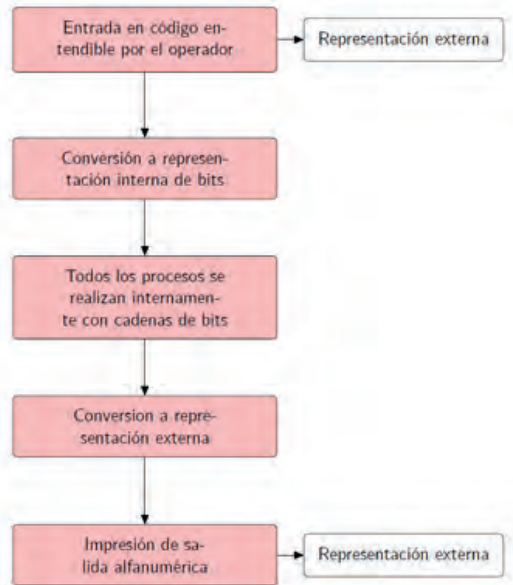


Figura 4.1: Procesamiento electrónico de datos.

Todo este proceso implica una codificación y posterior decodificación de la información.

Codificación

Codificar es aplicar un conjunto de reglas inequívocas que especifiquen la forma en que pueden representarse los datos, de forma tal que sea posible la decodificación ulterior.

Al margen de esta definición podemos decir que el propósito que tendría una codificación es posibilitar un manejo de la información que, de otro modo, no sería posible. Hoy, el hombre debe valerse de la codificación para decirle a una máquina qué es lo que quiere que haga, cómo, y con qué datos deberá manejarse. Como vimos, para ello utiliza el sistema binario, combinándolo y reformándolo con el objetivo de crear en base a él, distintas codificaciones que le permitan representar distintas cosas, sean números, letras, instrucciones, o estados internos de la máquina (señales de habilitación de un canal, de un periférico, etcétera).

La elección de un sistema de codificación adecuado es importante, ya que de esto dependerá la fiabilidad, universalidad y velocidad del sistema.

La velocidad de reconocimiento de la información varía según la potencia operativa del código. Por ejemplo, para transmitir una información necesitaremos una cantidad determinada de caracteres codificados, pero si ese sistema es menos potente, necesitaremos más caracteres con el consiguiente aumento en el tiempo de transferencia de los datos y de la posibilidad de que se pierda una porción de la información.

Existen en la actualidad una infinidad de códigos, algunos que son propios de una máquina o marca específica, y otros que son universalmente aceptados. Algunos ejemplos de estos códigos internacionales son los siguientes: BCD, EBCDIC, ASCII 8, etcétera.

La codificación consiste, entonces, en establecer una ley de correspondencia llamada código, entre las informaciones por representar y las posibles configuraciones binarias, de tal manera que a cada información le corresponda una y solo una configuración binaria.

Códigos

Representación de datos alfanuméricos

En el almacenamiento de un computador se pueden reconocer dos clases de datos: en forma de caracteres (es decir, alfanuméricos: nombres, direcciones, etcétera) y numéricos.

Dijimos que con grupos de 6 bits se podían representar $2^6 = 64$ combinaciones de bits, es decir, 64 símbolos diferentes. Tales códigos de 6 bits existieron, pero la mayoría de las computadoras de 3^{ra} y 4^{ta} generación trabajan con códigos de 8 bits. Con estos es posible representar los 36 caracteres alfanuméricos y símbolos más usados como \$, =, >, <, entre otros.

A cada grupo de 8 bits se lo denomina byte (del inglés *binary terms*), 1 byte = 8 bits. Los caracteres, entonces, se traducen a un código de 8 bits.

En informática hay dos códigos de uso general para representación de caracteres:

1. EBCDIC: *Extended Binary Coded Decimal Interchange Code*, que usan principalmente algunos modelos de IBM.
2. ASCII: *American Standard Code for Information Interchange*, de uso universal en microprocesadores.

Esquema de representación de un byte en ambos códigos: la representación de cada caracter se divide en dos partes, una de zona a la izquierda, y una, numérica, a la derecha.

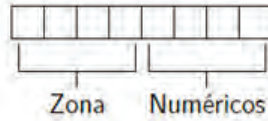


Figura 4.2: Zonas de un número de 8 bits.

1. ASCII

Este código utiliza 7 bits que proporciona $2^7 = 128$ combinaciones diferentes. El octavo bit o bit más significativo, es decir, el de la izquierda, es el bit de paridad; sirve para garantizar la conservación del contenido de un byte.

- Cada dígito (0 al 9) tiene 011 como la parte de zona del byte y su representación binaria como la parte numérica.
- Los caracteres alfabéticos (mayúsculas) están distribuidos en dos grupos según que sus bytes tengan 100 o 101 como la parte de zona.

Se adjunta la tabla de códigos ASCII de 7 bits; se utiliza “tal cual”, es decir, sin paridad, añadiendo un cero en la posición izquierda, o con paridad, añadiendo en esa posición el bit adecuado.

Existe también el código ASCII - 8; este código utiliza los 8 bits para almacenar un carácter y el computador almacena un bit extra o noveno bit llamado bit de paridad.

- Cada dígito (0 al 9) tiene 0101 como la parte de zona del byte y su representación binaria como la parte numérica.
- Los caracteres alfabéticos (mayúsculas) están distribuidos en dos grupos según que sus bytes tengan 1010 o 1011 como la parte de zona.

2. EBCDIC

El código ampliado decimal codificado en binario hace uso de 8 bits, presente principalmente en equipos IBM y compatibles.

Se forma:

- Cada dígito tiene 1111 como la parte de zona del byte y su representación binaria, como la parte numérica.
- Los caracteres alfabéticos (mayúsculas) están distribuidos en tres grupos, ya sea que los bytes tengan 1100, 1101 o 1110 como bits de zona.

Ejemplo de codificación EBCDIC.

CARÁCTER	ZONA	NÚMÉRICA
A	1100	0001
B	1100	0010
1	1111	0001
2	1111	0010

La codificación se comprende observando el siguiente diagrama

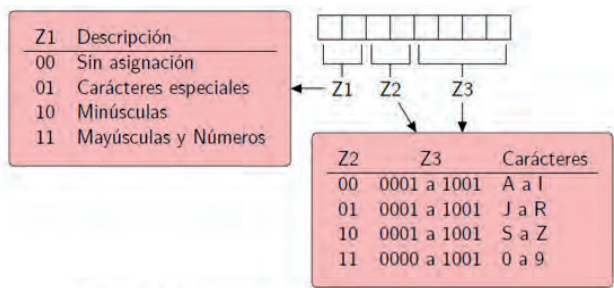


Figura 4.3: Partes de un número en formato EBCDIC.

Representación de datos numéricos

Representar números no es una operación sencilla y es preciso diferenciar varios casos. Los dispositivos de las computadoras en los que se efectúan las operaciones aritméticas solo podrán procesar números que les sean presentados según uno o más formatos bien definidos.

Existen dos métodos fundamentales para representar un dato numérico: codificar los dígitos decimales individualmente o codificar el número completo. En el último método tendremos, a su vez, dos tipos de formatos: los enteros de punto fijo y los números de punto flotante.

Codificación del número completo

1. Representación en formato de punto fijo

El primer componente de un formato es su longitud; el segundo, es el convenio escogido en la representación o codificación del número.

La representación en formato fijo es la manera más natural de escribir un número en una palabra de memoria.

Para representar enteros grandes, como veremos, necesitamos varios bytes, pero para hacer operaciones aritméticas con eficacia hay que emplear un número de bytes fijo, no variable; por tanto, la determinación del número de bytes supone la del mayor número representable.

Recordemos, además, que en electrónica no es posible representar directamente los signos “+”, “-”, sino que se deben utilizar ciertas convenciones. En todas ellas para un formato de trabajo de n bits (en general, 8, 16, 32, 64) se tiene que:

- El bit reservado para el signo es “0” si el número es positivo.
- El bit reservado para el signo es “1” si el número es negativo.

Forma binaria directa: para representar enteros puede usarse a la forma binaria directa, que es la representación decimal del número en el sistema binario. Con 8 bits pueden representarse directamente los números comprendidos entre 00000000 y 11111111, es decir, entre 0 y 255 de la base decimal. Se plantean dos dificultades que se detallan a continuación: la primera, que solo representamos números positivos; la segunda, que la magnitud de esos números queda limitada a 255, si trabajamos con solo 8 bits. A continuación, exploraremos las soluciones más comunes a estos inconvenientes.

Binario con signo: en un número representado en notación binaria con signo, según dijimos, este viene indicado por el bit de la izquierda llamado Bit Más Significativo (BMS), tradicionalmente “0” si es positivo, y “1” si es negativo; por lo tanto, el rango que se puede representar con 8 bits va desde:

$$\underbrace{1}_{-} \underbrace{1111111}_{127} = -127 \quad \underbrace{0}_{+} \underbrace{1111111}_{127} = +127$$

Figura 4.4: Bit más significativo

De esta forma ya se pueden representar números negativos, pero a costa de reducir la magnitud de 255 a 127. Además, se pierde un número ya que tenemos +0 y -0. Considerando el problema de la magnitud, para representar

números más grandes no hay más remedio que utilizar un número mayor de bits; es decir, usar una “palabra de memoria” mayor.

Se considera “palabra de memoria” a la longitud de la cadena de bits direccionada como un solo bloque en la memoria de la computadora (8, 16, 32, 64 bits). Utilizando, por ejemplo, palabras de 16 bits podemos representar todos los números comprendidos entre -32 Ki y $+32 \text{ Ki}$. (1 kbyte = 1024 bytes; $2^{15} \text{ bytes} = 32 \text{ kbytes}$). El bit 15 lleva el signo y los 15 restantes (los comprendidos entre el 0 y el 14) expresan hasta la magnitud $2^{15} = 32767$. Representación binaria:

Ejemplo: Representación binaria;

$$\begin{aligned}
 (+17)_{(10)} &= \underbrace{0}_{\text{signo} +} \underbrace{00\ 0000\ 0001\ 0001}_{17 \text{ en binario}} \\
 (-17)_{(10)} &= \underbrace{1}_{\text{signo} -} \underbrace{00\ 0000\ 0001\ 0001}_{17 \text{ en binario}} \\
 (+127)_{(10)} &= \underbrace{0}_{\text{signo} +} \underbrace{00\ 0000\ 0111\ 1111} \\
 (-129)_{(10)} &= \underbrace{1}_{\text{signo} -} \underbrace{00\ 0000\ 1000\ 0001} \\
 (+256)_{(10)} &= \underbrace{0}_{\text{signo} +} \underbrace{00\ 0001\ 0000\ 0000}
 \end{aligned}$$

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
±	Valor absoluto														

Figura 4.5: Partes de un número binario con signo de 16 bits.

Esta representación impone un tratamiento especial del signo y, por el mismo hecho, circuitos diferenciales para el procesamiento de sumas y restas, inconveniente que desaparece si se representan los números bajo la forma complementada.

Complemento a uno: la representación en binario se puede obtener de la siguiente manera:

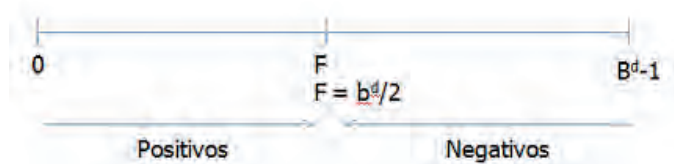


Figura 4.6

Dado un número entero, hallar el natural (la representación):

$$n = e + \delta(e) * b^d \quad \left\{ \begin{array}{l} \delta(e) = 0 \text{ si } e \geq 0 \\ \delta(e) = 1 \text{ si } e < 0 \end{array} \right.$$

Figura 4.7

Si queremos saber cuál es la representación (en 8 bits) del número 35:
 $e = 35$ y la función devuelve 0 porque es entero, es mayor que cero.

$$b_d = 256$$

Por lo cual, $n = 35$ y en binario será la presentación.

$$(+35) = 0000 \ 0000 \ 0010 \ 0011$$

Dado un natural, hallar el entero representado. Dado un número entero, hallar el natural (la representación).

$$e = n - \phi(n) * b^d \quad \left\{ \begin{array}{l} \phi(n) = 0 \text{ si } n < f \\ \phi(n) = 1 \text{ si } n \geq f \end{array} \right.$$

Figura 4.8

Si queremos saber cuál es el número entero representado (en 8 bits) en 0110 0010:

n (el decimal del número de la representación) = 98 y la función devuelve 0 porque la frontera es 127

$$b_d = 98$$

Por lo cual, $e = 35$ y en binario será la presentación.

$$(+35) = 0000\ 0000\ 0010\ 0011$$

Una manera más sencilla establece que en complemento a uno, todos los enteros positivos, se representan en su formato binario correcto. Para representar un número negativo se complementan cada uno de los bits de la representación original; es decir, se transforman todos los ceros en uno y los unos en cero.

Ejemplo:

$$\begin{aligned} (+3)_{(10)} &= 0000\ 0000\ 0000\ 0011 \\ (-3)_{(10)} &= 1111\ 1111\ 1111\ 1100 \\ (+127)_{(10)} &= 0000\ 0000\ 0111\ 1111 \\ (-127)_{(10)} &= 1111\ 1111\ 1000\ 0000 \end{aligned}$$

Complemento a dos: los números positivos se representan como binarios con signo exactamente igual que se hacía en complemento a uno. La diferencia estriba en la representación de los números negativos, que se hace determinando, primero, el complemento a uno y sumando uno a continuación; es decir, obteniendo el complemento auténtico.

Veamos un ejemplo:

$$\begin{aligned} (+10)_{(10)} &= 0000\ 0000\ 0000\ 1010 \\ (-10)_{(10)} &= 1111\ 1111\ 1111\ 0110 \\ (+125)_{(10)} &= 0000\ 0000\ 0111\ 1101 \\ (-125)_{(10)} &= 1111\ 1111\ 1000\ 0111 \end{aligned}$$

El complemento a dos constituye la forma de representación más adecuada para los procesadores más simples, como los microprocesadores. En

procesadores complejos puede recurrirse a otras formas de representación, como el complemento a uno, usando un circuito especial para corregir el resultado.

Cero desplazado: es otra técnica de punto flotante más sencilla.

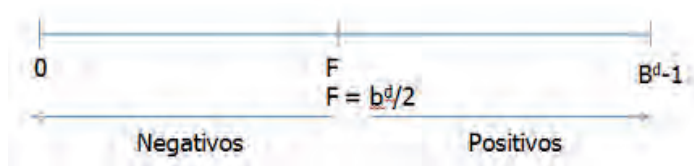


Figura 4.9

Para obtener el entero o el natural las fórmulas son las siguientes:

$N = E + F$, donde F es la frontera o el punto medio y se obtiene como $b^d/2$
 $E = N - F$

Si queremos saber cuál es el número entero representado (en 8 bits) en 0110 0010:

N (el decimal del número de la representación) = 98
 $E = N - F$, o sea, $E = 98 - 128$, entonces, el entero es $E = -30$

Ejemplo: Si queremos saber cuál es la representación del número 72
 $N = E + F$, o sea, $N = 72 + 128$, entonces, la representación del binario de 200 es:

$N = 1100\ 1000$

En la aritmética de punto fijo se le deja al programador el cuidado de situar la coma y se ve obligado a conocer y hacer evolucionar el lugar de la coma a lo largo de las operaciones. Por lo tanto, se usa la aritmética entera (punto fijo) cuando las operaciones se realizan con números que están restringidos a tomar valores enteros solamente. Cuando las operaciones se realizan con números que toman valores decimales, la aritmética usada es la de punto flotante.

2. Representación de punto flotante.

Los números fraccionarios decimales se pueden representar en almacenamiento, mediante su expresión en punto flotante; es decir, mediante sus representaciones en forma exponencial normalizada.

A continuación explicaremos cómo se realiza esto.

La condición básica es que los decimales deben representarse en algún formato fijo. Para no desperdiciar bits, la representación utilizada pasará por la normalización de todos los números.

Supongamos el número “0,000123”; al escribirlo, se desperdician 3 lugares correspondientes a los ceros a la izquierda del número, ceros que solo sirven para indicar la posición del punto. El número se normaliza haciendo: $0,123 \times 10^3$.

Es decir, el número se coloca en una escala tal que el punto decimal queda inmediatamente a la izquierda del dígito más significativo distinto de cero.

Ejemplo: Otros Casos:

Número	Notación exponencial normalizada
13,76	$0,1376 \cdot 10^2$
978,0	$0,978 \cdot 10^3$
0,00742	$0,742 \cdot 10^2$

En el número normalizado: $0,123 \times 10^3$, “.123” se llama **mantisa normalizada**; y -3 se denomina exponente. La normalización ha consistido en la eliminación de todos los ceros situados a la izquierda y en el cálculo del exponente. La forma general es:

$$SM \times \&^E$$

Donde:

- Indica el signo del número.
- Corresponde a la mantisa del número normalizado.
- Es el exponente del número normalizado.
- Es la base del sistema de numeración.

La notación que se usa en aritmética de punto flotante para representar internamente los números en el computador es, básicamente, una adaptación

de la notación científica. Por lo tanto, un número de punto flotante se compone de tres partes:

- El signo del número.
- Los dígitos del número, llamados *mantisa*.
- El *exponente* expresado como una característica de dos dígitos.

La figura muestra cómo están distribuidos los bits en el almacenamiento de un número binario de punto flotante, en forma de una palabra de 32 bits.



Figura 4.10: Partes de la representación de punto flotante para 32 bits

Como un exponente puede ser negativo, tenemos que incluir el signo del exponente en el campo reservado para él. Mejor que reservar un bit para el signo, empleamos la característica del exponente en lugar de su verdadero valor. Cuando se reserva un campo de 7 bits para el exponente, la característica de un exponente se obtiene agregando $2^6 = 64$ al exponente. Entonces $C = E + 64$. Esta forma de representar enteros se denomina de “Cero desplazado”. Exponente expresado con la técnica de cero desplazado.

Exponente verdadero	-64	-63	-62	...	-1	0	+1	...	+63
Característica	0	1	2	...	63	64	65	...	127

Cuadro 4.2: Exponente expresado con la técnica de cero desplazado.

Existen varios sistemas de representación de punto flotante. Es importante aclarar que las actuales técnicas utilizan palabras de 64 bits, pero a los fines prácticos utilizaremos palabras de 32 bits, veremos dos técnicas.

IBM-360

Una de las primeras técnicas de codificación de punto flotante es IBM-360.

Esta técnica divide la palabra de 32 bits de la siguiente manera:



Figura 4.11: Partes de la representación IBM-360

Para codificar el exponente se utiliza una técnica de punto fijo *cero desplazado* y para la mantisa una técnica de punto fijo de *Signo Valor Absoluto*.

Veamos un ejemplo del número a la representación:

Dado el siguiente número $123,456 \times 10^2$, cómo obtenemos la representación.

- 1- Resolvemos la forma científica para poder convertir a Hexa: 12345,6
- 2- Convertimos el número a Hexa con el método ya visto: 3039,9999
- 3- Normalizamos el número: $0,303999 \times 10^4$ (como la mantisa utiliza 6 dígitos Hexa, solo tomo los primeros 6).

Tengo el exponente, faltan calcular los dos primeros dígitos hexa de la representación:

$N = e + f$, entonces, $n = 4 + 64$, por lo cual, $n = 68$ que en binario en 7 bits es 100 0100.

El signo adelante es 0 por ser positivo, entonces, los 8 bits son 0100 0100, que en hexa es 44.

Por lo que la representación final es $(44303999)_{16}$

Veamos otro ejemplo de la representación al número:

$(C3AB7261)_{16}$

Tomamos los dos primeros dígitos hexa que tienen el signo y el exponente.

C3 en binario es 1100 0011, por lo cual, en signo es NEGATIVO.

El exponente será 100 0011 en decimal 67, o sea, $E = N - F$, por lo cual, $E = -61$.

Ahora hay que calcular el número de la siguiente manera:

$0,AB7261 \times (10)_{16}^{-61}, (10 \times 16^{-1} + 12 \times 16^{-2} + 7 \times 16^{-3} + 2 \times 16^{-4} + 6 \times 16^{-5} + 1 \times 16^{-6}) \times 16^{-61}$

Resolvemos la expresión y obtenemos el resultado que, recordemos, es Negativo.

PDP-11

Una técnica mucho más moderna y más utilizada es PDP-11.

Esta técnica divide la palabra de 32 bits de la siguiente manera:



Figura 4.12: Partes de la representación de PDP-11

Esta técnica usa un bit más para el exponente, lo cual permite un rango aún más grande de representación, esto lo logra porque a diferencia de IBM-360 que utiliza una base hexa para la representación, PDP-11 utiliza la base 2.

(A modo de ser más práctico, los ejemplos y ejercicios de PDP-11 se expresan en base 16, pero su fundamento es la base 2, que es fácil de convertir por ser potencias enteras una de la otra).

Entonces, al utilizar el sistema binario, todo número binario normalizado siempre, después de la coma, lleva un 1 (uno).

Veamos un ejemplo del número a la representación:

Dado el siguiente número 48, 654562 x 10⁴, cómo obtenemos la representación.

- 1- Resolvemos la forma científica para poder convertir a hexa: 486545,62
- 2- Convertimos el número a binario con el método ya visto: 1110110110010010001,10011. Debemos tratar de ver que, luego de normalizar, tengamos 24 dígitos binarios detrás de la coma.
- 3- Normalizamos el número: 0,111011011001001000110011 x (10)₂¹⁹. RECORDAR que el primer 1 después de la coma NO va.

Tengo el exponente, falta calcular los dos primeros dígitos hexa de la representación:

n = e + f, entonces, n = 19 + 128, por lo cual, n = 147, que en binario en 8 bits es 1001 0011.

El signo adelante es 0 por ser positivo, entonces, la cadena de bits es:

signo 0, exponente 1001 0011, y la mantisa sin el primer 1 es 11011011001001000110011.

Todo junto, pasado a hexa, queda $(49ED9233)_{16}$

Veamos otro ejemplo de la representación al número:

$(124EC82A)_{16}$

En este caso tomamos los tres primeros dígitos hexa que tienen el signo y el exponente.

124 en binario es 0001 0001 0100, por lo cual, el signo es positivo.

El exponente será tomado después del signo; es decir, los siguientes 8 bits 001 0001 0 en decimal 34, o sea, $E = N - F$, por lo cual, $E = -94$.

Ahora hay que calcular el número de la siguiente manera: el tercer bit hexa era el 4 en binario 0100, el primer dígito (0) forma parte del exponente, por lo que nos falta uno, ¿cuál? es el 1 implícito de la representación, por lo que queda 1100 y es E

0, $EEC82A \times (10)_2^{-94}$, $(14 \times 16^{-1} + 14 \times 16^{-2} + 12 \times 16^{-3} + 8 \times 16^{-4} + 2 \times 16^{-5} + 10 \times 16^{-6}) \times 2^{-98}$ Resolveremos la expresión y obtendremos el resultado que, recordemos, es un número negativo.

Ahora bien, en los ejemplos que hemos visto (cuando pasamos del número a la representación) observamos que los números a representar tienen en su forma científica un exponente chico; es decir, elevado a la -3, -2, -1 o 1, 2, 3.

Pero qué pasa cuando queremos, por ejemplo, representar valores tales como la masa del electrón 9×10^{-28} gramos o la del sol 2×10^{33} gramos, lo que supone una gama (rango) de números mayor a 10^{60} .

En estos casos no podemos resolver la forma científica como hemos visto, pues la cantidad de ceros sería enorme.

Nosotros partimos de la siguientes fórmula:

$M \times b^e = B^x$, donde:

M es la mantisa del número a convertir,

b es la base del número,

e es el exponente,

B es la base a la que se quiera pasar,

x es la incógnita, que será el exponente a la que se elevará la B.

Para lo cual, esa etapa la resolvemos usando el logaritmo natural.

x es nuestra incógnita $x = (\ln(M) + e \times \ln(b)) / \ln(B)$

Luego, se debe operar para hallar una mantisa que quedará multiplicada por la base B y un exponente entero. Habremos obtenido el exponente de nuestra representación. Solo quedará normalizar (de ser necesario y con la técnica que se utilice) y seguir los pasos ya vistos.

Veamos un ejemplo.

Si tenemos que representar $123,45 \times 10^{30}$

Por fórmula $x = (\ln(M) + e \times \ln(b)) / \ln(B)$, o sea,

$x = (\ln 123,45 + 30 \times \ln 10) / \ln 16$ (Base 2 si trabajo con PDP-11 o 16 en IBM-360)

$x = 29,73029692$. Volviendo a nuestra fórmula original, en su parte derecha $16^{29,73029692}$

Por propiedad de potencia puedo escribir como $16^{29} \times 16^{0,73029692}$;

si resolvemos la parte de exponente fraccionario, obtenemos $7,5746943 \times 16^{29}$;

y continuamos con los pasos descriptos del procedimiento natural; es decir, cambio de base a base 16, normalizo, etcétera.

Codificación de caracteres numéricos

Cada dígito del número es codificado individualmente por el código de caracteres del computador, por ejemplo, EBCDIC o ASCII-8, excepto que la zona del último dígito se reserva para el signo del número.

Los códigos de caracteres numéricos no se emplean para realizar operaciones aritméticas con los datos.

Estas cadenas de caracteres son generalmente de longitudes variables.

De acuerdo con esto, la mayoría de los computadores convierten al código de procesamiento más conveniente antes de llevar a cabo las operaciones aritméticas con los datos. Los resultados del procesamiento aritmético se vuelven a convertir, entonces, en el código de caracteres para la salida potencial.

La codificación del signo se refleja en la siguiente tabla y ejemplo:

Zona	Signo
1111	sin signo
1100	positivo
1101	negativo

Cuadro 4.3: Significado del 1/2 Byte de zona

Número	6	3	±7
+637	1111 0110	1111 0011	1100 0111
-637	1111 0110	1111 0011	1101 0111

Cuadro 4.4: Códificación en EBCDIC de los números +637 y -637.

Un código de procesamiento muy conocido es el sistema DECIMAL CODIFICADO en BINARIO o código BCD (*binary coded decimal*).

Representación BCD: esta técnica consiste en codificar cada una de las cifras decimales de un número mediante su representación binaria utilizando 4 bits, reservando para el signo el último campo de 4 bits.

Para codificar las diez cifras decimales comprendidas entre el 0 y el 9 hacen falta 4 bits (que dan dieciséis combinaciones).

Símbolo	0	1	2	3	4	5	6	7	8	9
Código	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Cuadro 4.5: Representación binaria de 4 bits de los dígitos decimales.

Como solo hacen falta 4 bits para codificar una cifra en BCD, pueden representarse dos cifras en cada byte y se forma el BCD empaquetado. Se utilizan los bits de zona reservados para el signo como últimos 4 bits (1100 para los positivos y 1101 para los negativos).

Retomando el ejemplo de ±637.

Número	6	3	7	±
+637	0110	0011	0111	1100
-637	0110	0011	0111	1101

Obsérvese que es posible convertir datos numéricos del código EBCDIC (o ASCII-8) de 8 bits en el BCD, mediante la eliminación de los unos (1) del medio byte izquierdo de cada dígito (bits de zona) y “empaquetando” o comprimiendo los bytes restantes.

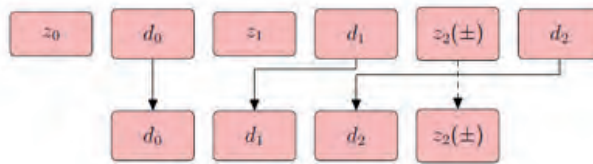


Figura 4.13: Empaquetamiento.

En general, en formato EBCDIC, en n bytes se puede representar un número de n dígitos decimales, mientras que, en el formato decimal empaquetado, en n bytes pueden representarse cifras de $2 \times n - 1$ dígitos decimales.

Control de paridad

Cuando la computadora transmite información de un lado a otro lo hace a altísima velocidad. Esto trae aparejado el riesgo de perder algún bit en el camino, esto ocurre cuando el receptor de la información recibe caracteres equivocados.

Para tratar de disminuir este riesgo se utiliza el control de paridad que funciona de la siguiente manera:

Se agrega un bit a la cantidad de bits que tenga el código utilizado. Por ejemplo, si trabajamos con el código EBCDIC vamos a tener el byte, conformado por 8 bits, más este bit adicional denominado de redundancia. El nombre se debe a que se produce una redundancia en la codificación, esto es, se agregan más bits de los necesarios para representar la información.

La función del bit adicional es detectar un error de transmisión, lo que se logra haciendo que la suma de todos los “1” de un byte sea par (paridad par) o impar (paridad impar).

De esta forma, si se opera con paridad par y en un byte a transmitir hay tres “1”, se enciende el bit de paridad (toma valor “1”), entonces, la suma es 4 (par).

Al contrario, si la suma de los bits del byte que se va a transmitir es par se deja el bit de paridad con valor “0” (cero).

De esta forma, al transmitir información el receptor puede tener el control de paridad, y si detecta que algún byte tiene una cantidad incorrecta de “1”, es porque hubo un error y solicita la retransmisión del mismo.

Este método detecta cuando hay un solo error en la transmisión de un byte, porque si son dos se compensan.

Si se debe transmitir el byte $N = 1111\ 0101$.

- Se debe notar que tiene 6 bits en “1”.
- Es necesario definir la ubicación del bit de paridad, para este ejemplo, primero enviaremos el bit de paridad (p), y luego N .
- Si se utiliza paridad impar es necesario que $p = 1$.
- Si optamos por *paridad par* usaremos $p = 0$.

Paridad	p	N	“1” transmitidos
Impar	1	11110101	7
Par	0	11110101	6

Cuadro 4.6: Paridad en números binarios.

Formatos de representación de números de punto flotante

Los siguientes cuadros resumen el proceso de representar e interpretar números de punto flotante en IBM-360 y PDP-11, con las siguientes consideraciones.

- Las variables σ , ε y μ corresponden al número natural representado por la cadena de bits indicada.
- Las variables S , E , M corresponden a los campos *Signo*, *Exponente* y *Mantisa* del número a ser representado (indicándose la base como subíndice).
- Las variables s , e , m corresponden a los campos *Signo*, *Exponente* y *Mantisa* del número interpretado a partir de la cadena de bits en cada formato.

- Al momento de calcular la variable m se realiza la división por 16^6 o 2^{24} . En ambos casos equivale a mover la coma antes del bit 24.
- En el caso particular del proceso mostrado en la figura [fig:InterpretacionMantisa], el valor de la mantisa ($\mu_{(16)}$) en ambos formatos serán iguales sí en el formato IBM-360 $X_1 \geq 8_{(16)}$ (es decir, que el bit 23 es 1).
- Se debe tener en cuenta que la misma cadena de bits representa números diferentes en cada formato.

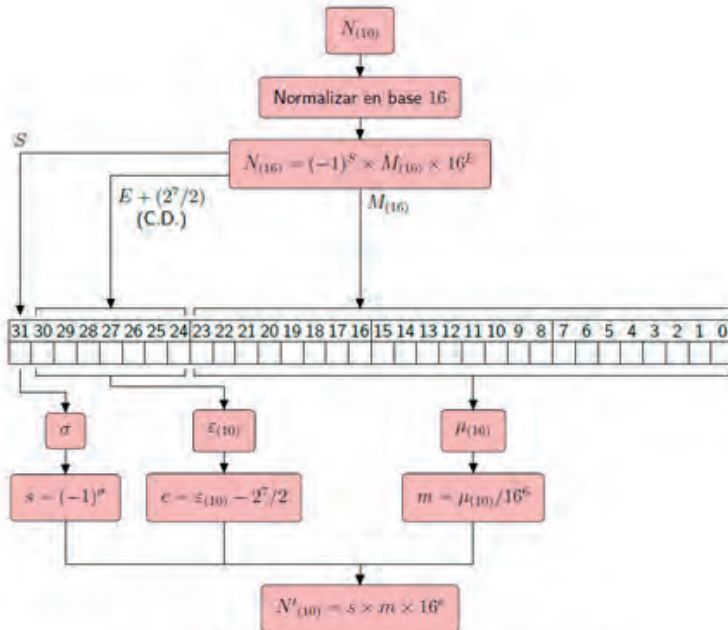


Figura 4.14: IBM-360. Representación e Interpretación.

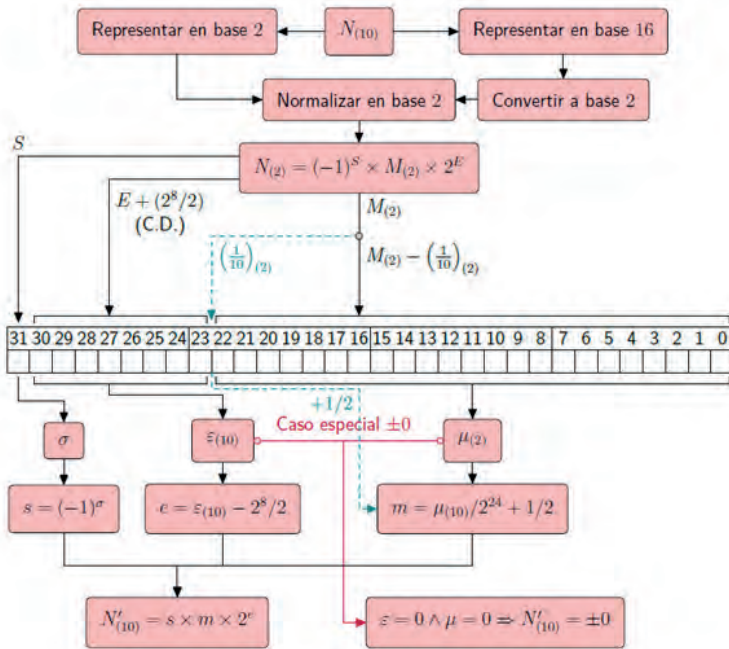


Figura 4.15: PDP-11. Representación e Interpretación.

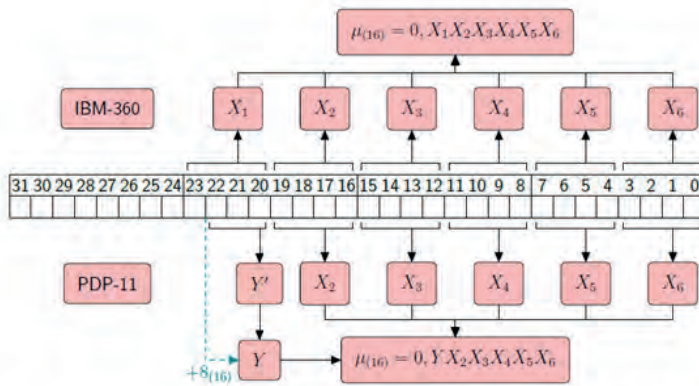


Figura 4.16: IBM-360 y PDP-11. Interpretar la mantisa usando base hexadecimal.

Procedimiento para representar en punto flotante

Usaremos el número $N_{(10)} = 153,48487 \times 10^2$ para dar un procedimiento para representar N en ambos formatos mencionados (IBM-360 y PDP-11).

$$\begin{array}{ll}
 N_{(10)} = (153,48487 \times 10^2)_{10} & (a) \\
 N_{(10)} = +1 \times (15348,487 \times 10^0)_{10} & (b) \\
 N_{(10)} = +1 \times (15348 + 0,487)_{10} & (c) \\
 N_{(16)} = +1 \times (3BF4 + 0,7CAC083 \dots)_{16} & (d) \\
 N_{(16)} = +1 \times (3BF4,7CA)_{16} & (e) \\
 \hline
 N_{(16)} = -1^0 \times 0,3BF47CA & \times 10^4 \quad (f) \\
 N_{(2)} = -1^0 \times 0,0011|1011|1111|0100|0111|1100|1010 & \times 10^{16} \quad (g)
 \end{array}$$

Es importante resaltar que el número N es siempre el mismo, solo es presentado en un formato o base diferente.

1. El número original es positivo y de exponente pequeño.
2. Desarrollamos la forma científica e identificamos el signo del número.
3. Se debe dividir el número en parte entera y fraccionaria, para su conversión.
4. Representar cada parte del número en base 16.
5. Obtener el número completo en base 16.
6. Normalizar la mantisa en base 16. Usar $(-1)^{\text{signo}}$ para el signo.
7. Representar en base 2 a partir de la representación en base 16. El signo es el mismo, la mantisa se transforma por el método de los agrupamientos, en el caso del exponente se debe tener en cuenta que $16 = 2^4$, por lo que $16^n = 2^{4 \cdot n}$.

Representación en el formato de IBM-360

Una vez que hemos normalizado N en base 16, podemos construir la cadena de bits de su representación (o la cadena de símbolos hexadecimales).

1. Primero debemos representar el exponente en el formato de cero desplazado de 7 bits (previa verificación de que sea representable). Aplicando la ecuación:

$$\begin{aligned}
 \text{natural} &= \text{entero} + \text{frontera} \Rightarrow n = e + f = e + 2^7/2 \\
 n &= 4 + 64 = 68 \\
 n_{\text{en 7 bits}} &= 1000100
 \end{aligned}$$

2. Completamos el primer byte con el bit de signo.

$$\text{Primer Byte a izquierda} = (\underbrace{0}_{\text{signo}} \underbrace{1000100}_{\text{exponente}})_{(2)} = 44_{(16)}$$

3. Completamos con los seis dígitos hexadecimales más significativos de la mantisa. Completando con ceros a la derecha de ser necesario.

$$N \text{ en IBM-360, en base 16} = 44|3B|F4|7C$$

$$N \text{ en IBM-360, en base 2} = 01000100|00111011|11110100|01111100$$

Representación en el formato PDP-11

De manera similar podemos partir de la expresión (g) para representar el número en formato PDP-11.

1. Es necesario asegurar que el número de la expresión (g) este normalizado en base 2.

$$N = -1 \times 0,0011|1011|1111|0100|0111|1100|1010 \times 10^{16}$$

$$N = -1 \times 0,11101111110100011111001010 \times 10^{16-2}$$

$$N = -1 \times 0,1110|1111|1101|0001|1111|0010|1000 \times 10^{14}$$

2. Una vez normalizado en base 2, debemos representar el exponente en el formato de cero desplazado de 8 bits. Aplicando la ecuación:

$$\text{natural} = \text{entero} + \text{frontera} \Rightarrow n = e + f = e + 2^8/2$$

$$n = 14 + 128 = 142$$

$$n_{\text{en 8 bits}} = 10001110$$

3. Completamos los primeros 9 bits a izquierda.

$$\text{Primeros 9 bits a izquierda} = (\underbrace{0}_{\text{signo}} \underbrace{1001110|0}_{\text{exponente}})_{(2)}$$

4. En el caso del ejemplo no es necesario, pero debemos completar la mantisa con ceros a la derecha hasta completar los 24 bits. El formato PDP-11 no codifica el primer “1” significativo pues está implícito en la técnica. Por lo que los bits en el campo de la mantisa son como los que se muestran a continuación:

1110|1111|1101|0001|1111|0010

5. Falta concatenar los 9 bits de la característica (el signo y el exponente) con los 23 bits de la mantisa.

N en PDP-11, en base 2 = $\overbrace{0}^{\text{signo}} \overbrace{100|111|0}^{\text{exponente}} \overbrace{110|1111|1101|0001|1111|0010}^{\text{mantisa}}$
 N en PDP-11, en base 16 = 4|E|6|F|E|1|F|2

Usando el logaritmo natural

Los procedimientos anteriores no definen ningún procedimiento matemático en particular para la conversión de N a la base hexadecimal o binaria. Los métodos más simples pueden resultar demasiado extensos (y propensos a tener errores de cálculo), por ello, se aplica $\ln$ para generar una forma de N más útil.

El $\ln$ permite generar la representación $N = -1^S \times B^x$.

$$\begin{aligned}
 N &= -1^S \times M \times 10^E \\
 -1^S \times B^x &= -1^S \times M \times 10^E && \text{Eliminando el signo} \\
 \ln(B^x) &= \ln(M \times 10^E) && \text{Aplicando } \ln \text{ a ambos miembros} \\
 x \times \ln(B) &= \ln(M) + E \times \ln(10) && \text{Aplicando propiedades de } \ln \\
 x &= \frac{\ln(M) + E \times \ln(10)}{\ln(B)} && \text{Despejando } x
 \end{aligned}$$